

Comparative Effectiveness of StatsClaw and Base Claude Code on Financial Research Replications

FINM 33200 Final Project Report

By Charlie Carvajal, Cole Ginter, Jonathon Nie, and Alex Nikolaev

1. Introduction and Methodology

For our final project, we sought to conduct original research on agentic AI by evaluating the performance of StatsClaw against base Claude Code on reproducing research in finance. The evidence we evaluate is agent use cases, where we compare the performance of both modes and analyze the unique and common mistakes in them. We evaluate StatsClaw's and Claude Code's performance by replicating an easy case: the Federal Reserve Treasury Yield Curve from *Gurkaynak, Refet S., Brian Sack, and Jonathan H. Wright. 2007*, a medium case: Credit Default Swap Portfolios based on *He, Zhiguo, Bryan Kelly, and Asaf Manela. 2017* and *Palhares, Diogo. 2013*, and a hard case: Tables 2 and 3 from *He, Zhiguo, Bryan Kelly, and Asaf Manela. 2017*. Prior to running our experiment, we expected to find that both modes can perfectly complete the easy case, and neither was able to fully complete the medium and hard case. To our surprise, the reverse was true. StatsClaw outperformed base Claude Code for the easy case, but Claude Code outperformed StatsClaw for the medium and hard case. In the medium case, neither framework was able to fully replicate the oracle, but the failure modes differed: StatsClaw correctly identified the return formula but silently violated the prompt's discount curve instruction and missed two of the three required data sources, while base Claude raised the discount curve ambiguity as an explicit decision point and identified all three data sources correctly. Similarly, while neither framework was able to replicate the oracle, base Claude outperformed StatsClaw more decisively in the hardest case. These results suggest that StatsClaw rigorous methodology is more conducive to perfectly replicating simpler pieces of literature rather than soundly attempting more complicated replications.

We took a two-pronged approach to evaluating the performance of these frameworks. First, the simple check: how closely do the numbers match the oracle being replicated? However, particularly for our medium and hard cases, we knew it was unlikely that the StatsClaw and base Claude Code would be able to fully replicate the tables, so we made sure to have the frameworks store the key assumptions and decisions made. Afterwards, we evaluated the quality of these choices and whether they violated the prompt in any way. Because the purpose of this project is to test whether these frameworks could perform these replications independently, the users did not provide additional assistance that perhaps would have been expected if the model and researcher were working collaboratively. Therefore, a key piece of this second pronged evaluation was to see what mistakes or poor decisions were made without revealing them to the user. These types of mistakes were seen as particularly unfavorable because even if we had been collaborating with the frameworks they would have likely still occurred unbeknownst to the user. This highlights our general approach to quantitatively and qualitatively evaluating the frameworks' performances.

Another key methodological choice we made that is key to replicating our results is that the base Claude Code and StatsClaw replications were done on separate Claude accounts to prevent information leakage. As was discussed throughout the course, one of the key challenges when testing AI agents' abilities is preventing data contamination. Claude provides features such as conversation memory, search across past chats, and analysis on user preferences from previous conversations. Because these features can be implemented invisibly, using a single account to do both frameworks' replications could result in information from whichever arm is done first being leaked into the second replication without the user knowing. Isolating these replications is particularly important for the CDS returns replication and the He, Kelly, and Manela replication, as both of those have methodological ambiguities that the framework only uncovers through iteration, however, if the framework already knows the result of that iteration, it could unfairly skip to the answer. Performing these replications on separate accounts provides as close to an unbiased replication as we feel is possible for this type of survey.

This section highlights the goal and our approach to the project. The subsequent sections will expand on how StatsClaw uniquely approaches replicating papers, the results of each of the case studies, as well as a summary of our results and their limitations.

2. StatsClaw Overview

Before continuing, we should explain how StatsClaw works. StatsClaw is a workflow framework for Claude Code designed to develop statistical packages and distributed through a public GitHub repository. Therefore, StatsClaw is not a separate model or API but rather a way to configure Claude Code such that it behaves as a coordinated team of specialized agents instead of a single agent in the base Claude Code format. The central idea behind StatsClaw is that the "team" of agents works through adversarial verification. For the replication of statistics papers, this takes the form of a builder agent producing results that must pass the tests independently created by a tester agent and then align with a Monte Carlo simulation independently created by a simulator agent. However, this leads to an impactful distinction between statistical and financial research: all three of our (financial) replications used real-data instead of simulated data, which means instead of three sources of independent adversarial verification, we only had two. Therefore, we would expect, and from what was revealed from our cases, that the decrease in adversarial verification increases the number and degree of inaccuracies.

Beyond the adversarial agents, StatsClaw creates several other agents to execute the pipeline. The leader agent delegates the tasks of the other agents and ensures that there are strict information barriers between agents. The planner agent reads the replication documents and then extracts the data sources, methodology, and edge cases. The planner then creates *spec.md* for implementation specifications for the builder and *test-spec.md* for validation specifications for the tester. These documents are kept isolated from each other such that the tester cannot see the *spec.md* and the builder can not see the *test-spec.md*. The scribe agent

documents the steps and assumptions of the replication in *ARCHITECTURE.md*, *evaluation.md*, and other audit documents to ensure transparency and reproducibility. The reviewer agent ensures convergence between the builder and test agent and the shipper agent upon confirmation from the reviewer commits the final state to the target repository.

To conclude our overview of StatsClaw we will discuss how we downloaded and implemented the framework. We began by copying the GitHub repository and opening it in VS Code. Then within the same workspace, but outside of the StatsClaw folder we create a target repository. From the StatsClaw repository we copy over several key files and folders. We put in a *CLAUDE.md* file which gives the agent teams framework to Claude Code, an agents folder which defines the roles of each agent, a skills folder which gives the agents a base set of skills to use, and profiles folder which provides the abilities of several common coding languages, a templates folder which provides scaffolding for the repository, and *.claude/settings.json* which is similar to a config file and gives Claude Code base allowances.

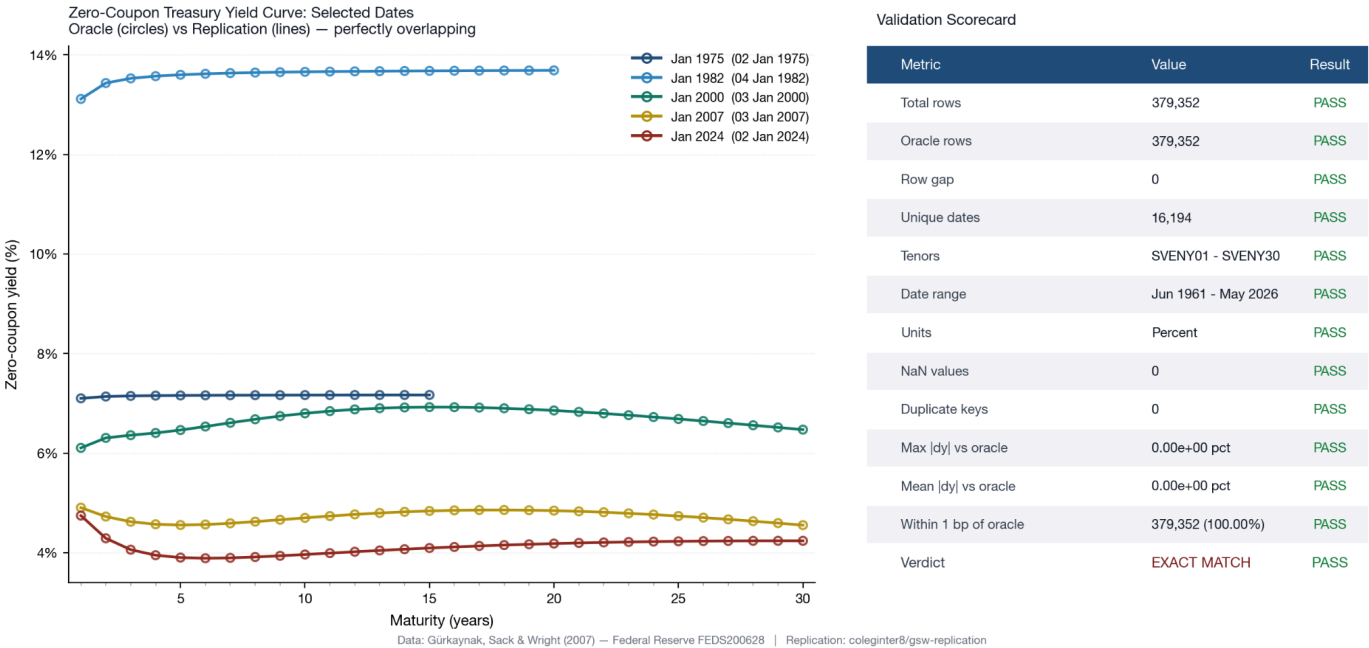
3. Case 1: Federal Yield Curve

The first case we used was a replication of the zero coupon yield curve data in the *fed_yield_curve* folder within the Financial Time Series Forecasting Repository. This repository used the methodology established in *Gurkaynak, Refet S., Brian Sack, and Jonathan H. Wright. 2007* to estimate the U.S. Treasury zero-coupon curve. However, the key insight is that the repository does not estimate the yield curve from raw treasury bond prices but rather pulls the data from the Gurkaynak, Sack, and Wright dataset published by the federal reserve. The repository then processes the data so that it can be used in other time series projects.

We choose this repository as our “easy” case in order to establish a floor for both StatsClaw and base Claude Code. The reason this is considered our easy case is because all the framework had to do was find the already published data and clean it. This requires little methodology and assumption, ensuring both frameworks are at least minimally useful for replicating financial research. Therefore the key idea being tested was whether the framework would realize that it did not need to download and shape the Svensson curve from scratch but rather find the directly estimated data from the Federal Reserve. Beyond this test, we also evaluated whether the frameworks could effectively process the data which included the appropriate handling of NA’s and matching dtypes. Below was the exact prompt used for the StatsClaw replication:

Construct the GSW (2007) daily zero-coupon Treasury yield curve dataset. Paper is at papers/gsw_2007.pdf. Validation oracle is at validation/validation_oracle.parquet — the tester may read it; the builder must not. Target repo is [username]/gsw-replication, workspace is [username]/workspace. Deliverable: a single parquet file with columns ds, unique_id (SVENY01–SVENY30), y (yield in percent), daily frequency, maximum coverage. Maintain evaluation.md. Start with the planner.

As expected, StatsClaw, using *Gurkaynak, Refet S., Brian Sack, and Jonathan H. Wright. 2007* paper alone, was able to successfully identify the Federal Reserve as the source of the data rather than downloading and calculating the results from scratch. All nine of the validation tests were passed without modification and the maximum absolute difference between the variables was 0.000000. Additionally, all missing values were correctly handled and all data types matched the table produced in the original repository. In total to accomplish this StatsClaw took approximately 33 minutes and used approximately 432 thousand tokens. Below is a figure that shows the exact overlap of the StatsClaw replication with the original data as well as the passed verification scorecard.



Overall this case demonstrates StatsClaw’s ability to successfully replicate financial research when the methodology is completely specified and the data source is published publicly.

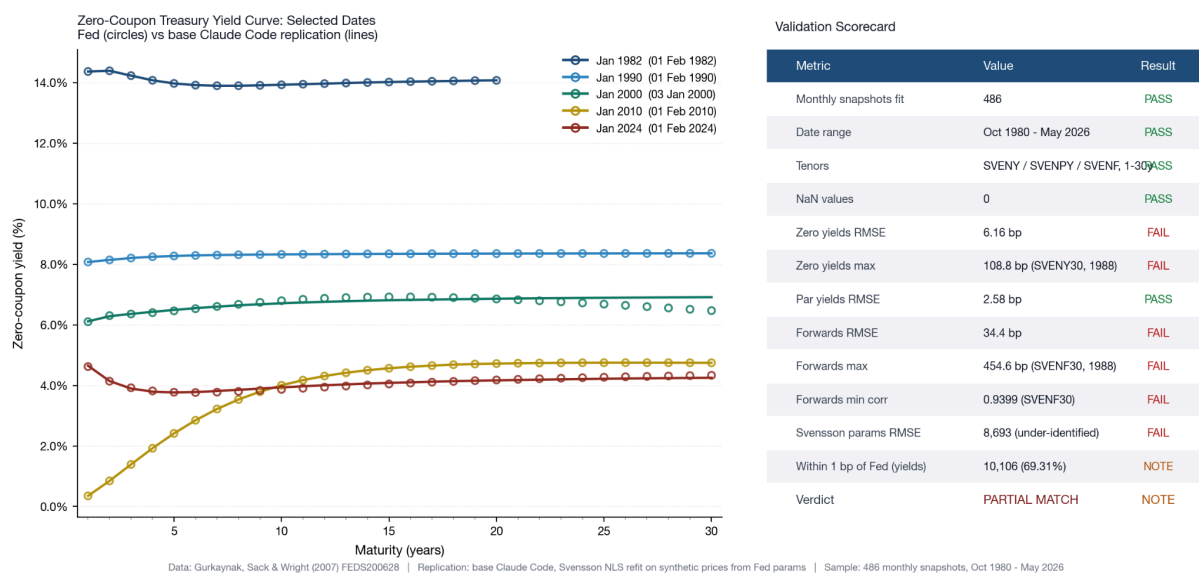
The base Claude Code replication, in contrast, did not identify the Federal Reserve's published FEDS200628 dataset as the intended shortcut. Instead, it implemented the full Svensson estimation pulling the CUSIP master for nominal Treasury notes and bonds from the U.S. Treasury Fiscal Data API, applying the paper's eligibility filters (excluding callable bonds, TIPS, FRNs, and on-the-run/first-off-the-run issues, requiring ≥ 90 days to maturity, excluding 20-year bonds from 1996 onward), pricing each eligible bond as a discounted cash-flow stream on a Svensson zero curve, and recovering parameters via duration-weighted nonlinear least squares. Because CUSIP-level historical Treasury bond prices are not publicly redistributable, base Claude Code constructed a self-consistency simulation: it used the Fed's own published Svensson parameters to generate synthetic clean prices for each eligible bond on each settlement date, then re-fit the curve to those simulated prices and compared the recovered yields back to the Fed file. We ran this pipeline over the maximum window the Fiscal Data API supports: first business day of each month from October 1980 (the earliest

date with a bond universe large enough) through May 2026, producing 486 monthly snapshots spanning roughly 45 years of yield-curve history.

Below was the exact prompt used for the Claude code replication:

replicate the dataset and paper located in fed_yeild curve replication/200628pap-2.pdf, reconstruct the dataset in the paper using tools or intrinsic knowledge and run the target evaluation script called evaluate_replication.py to test the effectiveness of the replicated result.

Running the evaluation script on these 486 monthly snapshots give us a good top-down view. In the 2010s, a decade of smooth, low-volatility yield curves, the replication is effectively exact: the zero-coupon yield curve RMSE across SVENY01–SVENY30 is 0.11 basis points, the maximum absolute error across the decade is 2.04 bp, and 99.6% of all yield cells fall within 1 bp of the Fed file. The 2020s subsample is similarly tight (RMSE 2.63 bp, max 15.5 bp). The 1980s, 1990s, and 2000s, however, reveal substantial divergence at the long end of the curve during high-volatility episodes. The worst dates are concentrated in early-to-mid 1988 in the aftermath of the 1987 crash, with maximum SVENY30 errors of 108.8 bp and SVENF30 errors reaching 454.6 bp; the January–February 1983 high-yield episode contributes the largest 20-year forward errors (around 290 bp); and four months in 1999 contribute SVENF30 errors above 200 bp. Aggregated across the full sample, the zero-coupon yield curve has a group RMSE of 6.16 bp (failing the 5 bp default threshold), the par-yield curve has RMSE 2.58 bp (passing the 5 bp threshold but failing the 25 bp max-error threshold at SVENPY30), and the instantaneous-forward curve has RMSE 34.4 bp with a maximum of 454.6 bp at SVENF30 (failing both default thresholds). Across the full sample 69.3% of yield cells fall within 1 bp of the Fed file, and the run fails 8 of the script's default threshold checks overall, in contrast to a near-pass on all checks for the 2024 sub-sample alone. Below is a figure showing the recovered zero-coupon curve at five snapshot dates spanning 1982 to 2024 alongside a scorecard summarizing the full-sample evaluation.



The two replications reach a similar yield curve on completely different routes. StatsClaw's planner agent read the paper, recognized that the Federal Reserve already publishes the fitted curve as a downloadable CSV, and treated the task as a download and reshape problem, producing an exact match against every cell of the oracle at every tenor in every regime. Base Claude Code, by contrast, reimplemented the full Svensson estimation pipeline whose output the Fed already publishes, and the quality of that pipeline's output is now visibly time dependent. In the smooth-curve era of the 2010s the refit recovers the Fed series to within roughly 0.1 bp; yet during volatile or high-curvature episodes, most acutely 1988, but also 1983 and 1999, the long end of the curve diverges by tens to hundreds of basis points. These long-end errors are not a defect of the implementation but a structural consequence of the refit approach. With an off-the-run bond universe that does not always reach 30 years in the earlier decades, and a six-parameter Svensson form whose curvature loadings are poorly identified, the recovered parameters can match the Fed's curve to within a basis point at the maturities where bonds exist while diverging at the maturities where the model is extrapolating. StatsClaw's replication is therefore more reliable with less code, no dependency on synthetic bond prices, and an exact match against the oracle in every period.

4. Case 2: Credit Default Swap Returns

The next piece of financial research we use as a replication test is the `cds_returns` repository also within the Financial Time Series Forecasting Repository. This pipeline used methodology established in *He, Kelly, and Manela (2017)* and *Palhares (2013)* to construct monthly credit default swap returns. The repository pulls Single-name CDS spreads from WRDS Markit CDS, the Zero-coupon yield curve from the Federal Reserve, and Short-term Treasury yields (3M, 6M) from FRED. The pipeline then uses this data to produce a parquet of 20 portfolios one of four tenors by one of five quintiles as well as a parquet that contains the Individual contract-level returns. The goal is therefore to replicate these two datasets.

We used this financial repository as our medium difficulty case because unlike the first case where there was one main evaluation metric, this case requires the frameworks to overcome two, more-complex, replication challenges. The first is a data challenge: the frameworks have to pull data from three sources, one of which is a complicated WRDS query that requires several filtering choices. The second is a difficult methodology because the frameworks must be able to correctly apply the two separate academic papers to calculate the CDS returns. Therefore this builds on the first test case because instead of pulling and cleaning from a public datasource, the frameworks must be able to correctly query for the right data and then apply the correct methodology to them. The exact prompt we used to generate this replication in StatsClaw was:

Replicate the HKM (2017) CDS portfolio returns pipeline using the Palhares (2012) mark-to-market return methodology. Papers are at papers/hkm_2017.pdf and papers/AQR CashFlow Maturity and Risk Premia in CDS Markets.pdf. Validation oracles are at validation/validation_portfolio.parquet (20 portfolios) and

validation/validation_contract.parquet (individual contracts) — the tester may read both; the builder must not. Target repo is [username]/cds-replication, workspace is [username]/workspace. WRDS credentials are in ~/.pgpass.

Deliverables:

ftsfr_cds_portfolio_returns.parquet — columns: ds (month-end date), unique_id ({tenor}_Q{quintile}, e.g. 5Y_Q1), y (return) — 20 portfolios (4 tenors × 5 quintiles), monthly

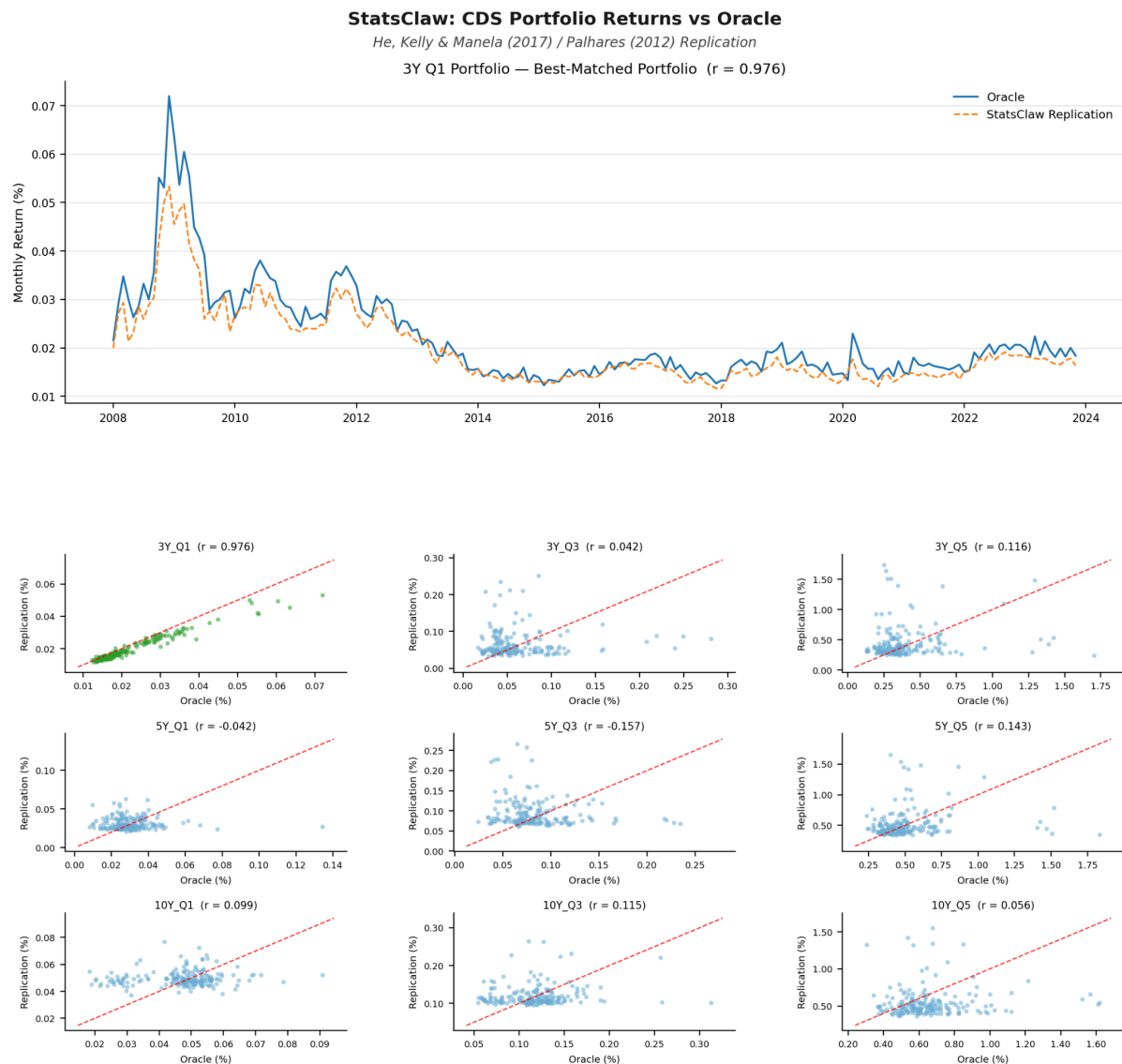
ftsfr_cds_contract_returns.parquet — columns: ds, unique_id ({ticker}_{tenor}), y — individual contract returns, monthly If the methodology requires a discount curve or short-rate input, raise a HOLD and ask rather than assuming. Maintain evaluation.md. Start with the planner.

The results of this test were ambiguous. After four iterative builder runs the validation requirements had to be relaxed mostly attributed to issues finding and filtering the data. Beginning with what worked, StatsClaw correctly identified the protection-seller monthly return formula from *He, Kelly, and Manela (2017)* and *Palhares (2012)*:

$r_t = S_{t-1}/12 + (S_{t-1} - S_t) \times RD_{t-1}$. Through this formula StatsClaw was able to match the sign convention of positive returns when spreads tighten and negative returns when spread wide. This led to the output file having a 100% match rate against the validation oracle. Additionally all of the portfolios behave in a way consistent with risk-compensation predicted by both papers. Particularly, mean returns increase monotonically across the four tenors and five quintiles. It is worth mentioning that the 3Y_Q1 portfolio was the only portfolio to surpass the initial correlation threshold of 0.90 with a correlation with the oracle of 0.976. This was the only portfolio to achieve this, which could be due to (as an ex-post rationalization) the framework being able to most successfully apply the methodology for portfolios dominated by macro factors across universe choices. But since this was the only portfolio to accomplish this and we did not have that expectation going in we will not count it as an overall success. Lastly, and perhaps most importantly, StatsClaw did apply an appropriate HOLD when it realized it was unable to independently shrink the number of tickers to match the numbers in the oracle. While the purpose of this study was to see how far StatsClaw could go independently, had we sought to use it more collaboratively we could have given clearer instructions which would have resulted in a stronger result. In other words, this replication likely would have been possible had we been collaborating with StatsClaw as the framework was designed, instead of testing its abilities.

This segues well into the two clear failures of this replication. The first, which is somewhat counter to the last point made in the success paragraph, is that StatsClaw did directly violate the user prompt. In particular, a HOLD never occurred to address the following sentence: “If the methodology requires a discount curve or short-rate input, raise a HOLD and ask rather than assuming.” Despite these instructions, in section 3.2 of the *spec.md* file the framework assumes the zero risk-free rate to be 0 pre-2008 and post-2008 use Markit’s pre-computer riskypv01. Both are clear assumptions about the discount curve that were not checked with the user. This, perhaps, represents a flaw in the adversarial verification process; because the

planner created the *spec.md* file and the builder executed it, parts of the prompt got lost in the translation and no agent was able to check it. This resulted in a direct non-compliance with the user prompt. The second failure was in data sourcing. The cds-return repository uses three data sources: WRDS Markit CDS, the Federal Reserve, and FRED, however StatsClaw only used WRDS Markit CDS. This is what likely contributed to the mismatch in tickers discussed in the previous section. But, as mentioned before, had we been collaborating with StatsClaw instead of testing it, we could have pointed the framework in the right direction instead of letting it continue to make the mistake of only using WRDS Markit. Below is a summary that analyzes the strengths and weaknesses of the StatsClaw replication.



These results show that, in the end, StatsClaw was not able to accurately replicate the results in the oracle, but do highlight significant methodological achievements. It is our belief that had we worked collaboratively with StatsClaw this replication would have been possible.

The base Claude Code replication of the CDS portfolio returns pipeline was run on a separate Claude account using plan mode, following the same isolation approach described in Section 1. The prompt passed to Claude was:

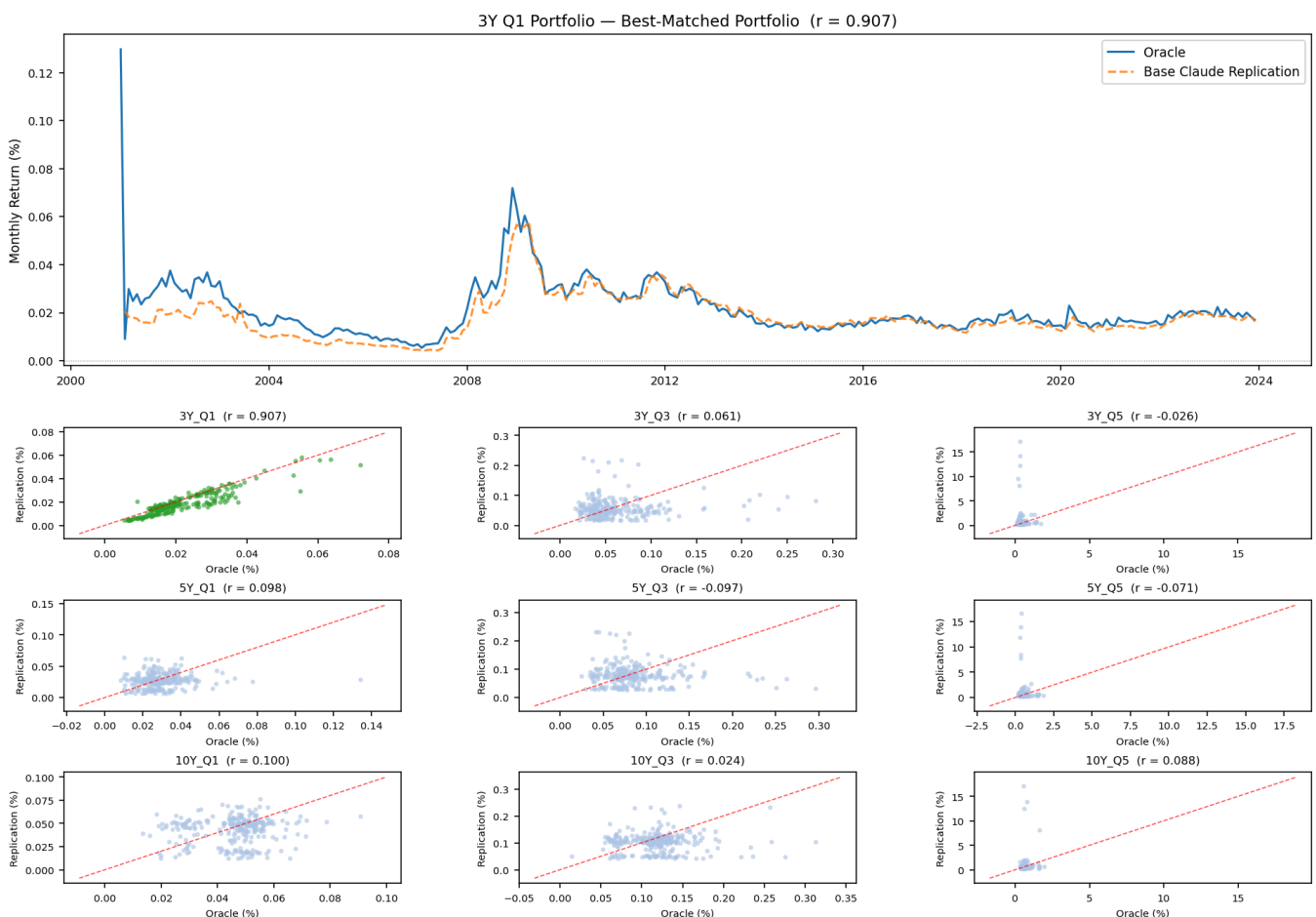
Replicate the HKM (2017) CDS portfolio returns pipeline using the Palhares (2012) mark-to-market return methodology. Papers are hkm_2017.pdf and AQR CashFlow Maturity and Risk Premia in CDS Markets.pdf. Validation oracles are validation_portfolio.parquet (20 portfolios) and validation_contract.parquet (individual contracts). WRDS username is: "[username]" and WRDS password is in ~/.pgpass. Deliverables: ftsfr_cds_portfolio_returns.parquet — columns: ds (month-end date), unique_id ({tenor}_Q{quintile}, e.g. 5Y_Q1), y (return) — 20 portfolios (4 tenors × 5 quintiles), monthly. ftsfr_cds_contract_returns.parquet — columns: ds, unique_id ({ticker}_{tenor}), y — individual contract returns, monthly. If the methodology requires a discount curve or short-rate input, raise a HOLD and ask rather than assuming. Create a .md file keeping track of every decision you make during your replication process.

Plan mode forced the agent to write out its full approach before generating any code, and this produced a detailed methodology_decisions.md file logging every formula choice, data source, and validation result. This documentation proved essential for post-hoc auditing of what the agent did and why. While in plan mode, Claude surfaced a user prompt asking "Which risk-free term structure source should I use for the risky duration computation?", to which the selections, GSW + swap rates (HKM spec), Optionmetrics (Palhares spec), and Treasury-only (simplified), were provided. Claude was instructed to select whichever one it thought optimal, it selected the Optionmetrics option, then continued to formulate its plan document.

Beginning with what worked, the discount curve was correctly resolved via the plan mode interaction described above. Notably, Claude correctly recognized that the choice was ambiguous between the HKM and Palhares specifications and surfaced it as a decision point rather than silently assuming, in contrast to StatsClaw's failure in this same case, which assumed a zero risk-free rate with no documentation. Once Claude specified using the Optionmetrics source, it correctly implemented the zero-coupon rates from WRDS (optionm_all.zerocd) throughout the pipeline. The agent also correctly applied the Palhares (2013) seller-of-protection return formula: $r_t = S_{t-1}/12 + (S_{t-1} - S_t) \times RD_{t-1}$, with the carry term based on the lagged spread and a capital gain of $(S_{t-1} - S_t) \times RD_{t-1}$ using the lagged risky duration, yielding positive returns when spreads tighten. The doc-clause priority ordering (XR14 > XR > MR14 > MR) was applied correctly to deduplicate the Markit CDS data, reflecting the shift from MR to XR conventions that occurred for North American IG CDS around 2009. At the contract level, 97.4% of observations fell within 5% of the oracle, with a correlation of 0.805 on that subset. The 2.6% of outlier observations correspond to credit-event or extreme-spread-jump months that the oracle handles differently.

The main failure was at the portfolio level. Base Claude constructed 20 portfolios (4 tenors $\times$ 5 spread quintiles) but only the 3Y_Q1 portfolio achieved a meaningful correlation with the oracle (0.907). The remaining 19 portfolios showed near-zero correlations. This failure traces to a single root cause: a universe mismatch. The pipeline pulled all USD senior foreign CDS from Markit, producing 4,369 unique tickers, while the oracle appears to use approximately 1,683 tickers filtered by criteria not stated in the papers, likely a combination of minimum credit quality, data completeness requirements, or domicile restrictions. Without knowing the oracle's exact membership filter, it was not possible to recover matching quintile boundaries. The agent documented this limitation explicitly, noting that the oracle universe is not recoverable from raw Markit data alone. A related issue was that the agent discovered empirically, rather than from the paper, that portfolio returns must be carry-only rather than full mark-to-market. The oracle portfolio returns are uniformly non-negative across all 5,510 observations, which is impossible under full MTM returns when spreads tighten. The agent inferred the carry-only formula from this property of the oracle, which is an example of the agent correctly using the validation data to converge on the right methodology, but also highlights that the paper alone was not sufficient to unambiguously specify this choice.

Base Claude Code: CDS Portfolio Returns vs Oracle
He, Kelly & Manela (2017) / Palhares (2012) Replication



Comparing the two arms, both StatsClaw and base Claude failed to replicate the portfolio-level oracle beyond 3Y_Q1. However, the failure modes differ in an instructive way. StatsClaw failed because it only used WRDS Markit and missed the FRED and Federal Reserve data sources entirely, which contributed to its universe mismatch. Base Claude correctly identified all three data sources specified in the original repository (WRDS Markit, Optionmetrics, and FRED) and used them, but still could not narrow to the oracle's exact universe. StatsClaw also violated the prompt's explicit HOLD instruction regarding the discount curve by silently assuming a zero rate; base Claude instead correctly identified the discount curve choice as ambiguous and raised it as an explicit decision point in plan mode, allowing the user to specify the correct specification before any code was written. In that sense, base Claude's methodology was more faithful to both the papers and the user's prompt, even though neither framework ultimately produced a full portfolio-level match.

5. Case 3 HKM Tables 2 and 3

Our final case involves replicating Tables 2 and 3 from *He, Kelly, and Manela (2017)*. Table 2 shows the relative size of primary dealers to broker-dealers, banks, and all Compustat firms across their total asset, book debt, book equity, and market equity over three time periods 1960-2012, 1960-1990, and 1990-2012. Table 3 is split into two panels. Panel A shows the correlation of Market capital, Book capital, and AEM leverage against each other as well as five macro factors. Panel B is of the correlations of the same variables as the factors/growth rates instead of their levels. According to the paper, the data should be sourced from WRDS CRSP, CRSP-Compustat, and Datastream. The primary trap when assembling this data is to find the holding companies for the primary dealers mentioned in the paper, instead of using the primary dealers directly.

We considered this case the hardest replication for several reasons. First, as previously mentioned, the paper constructs the capital ratio using the holding companies of the primary dealers listed in the New York Fed's list of primary government securities dealers. Therefore, to accurately perform this replication, the framework must map the subsidiary dealers in the New York Fed list to their primary dealers at each point in time, which can change over time. The second challenge is a data vintage problem. Compustat backfills and edits its historical data, so the data being pulled today will not be able to exactly match the data pulled by He, Kelly, and Manela in 2016. From previous examination, it is clear that these backfills and edits lead to significantly different results. Therefore, a key part of this examination is interpreting how StatsClaw acts when an exact replication is truly impossible. Thus, the primary challenge in this case is finding the best possible data and then concluding that an exact match is not possible. Below is the prompt we used to generate the replication:

Replicate Tables 2 and 3 only from He, Kelly & Manela (2017), "Intermediary Asset Pricing: New Evidence from Many Asset Classes," JFE 126: 1–35. Paper is at hkm-paper.pdf. Validation oracle is at validation/validation_oracle.parquet — the tester may read it; the builder must not. Target repo is [username]/hkm-replication, workspace is [username]/workspace. WRDS credentials are in ~/.pgpass. No pre-specified GVKEYs or

PERMNOs — identify primary dealers by name-matching the paper's appendix. Use a Z.1 aggregate splice for the 1970–1977 pre-WRDS gap. Maintain evaluation.md. Start with the planner.

The results of this case were pretty much a universal failure. While the failures greatly outweigh the successes, it is still worth mentioning what went right. Similar to the `cds_returns` case, StatsClaw was able to access WRDS effectively. Additionally, StatsClaw was able to pull the CRSP-Compustat linktable with the correct filters (linktype IN ('LU', 'LC', 'LS'), linkprim IN ('P', 'C'), and linkdt <= date <= linkenddt). Methodologically, the formula for the AR(1) capital factor and residual factor is the most mathematically intense, but statsClaw was able to successfully run the regression: $\eta_t = \rho_0 + \rho \times \eta_{t-1} + u_t$ and extract the appropriate residuals u_t/η_{t-1} . However, beyond this, there was little that could be pointed to that was fully accurate in this replication.

Firstly, as mentioned before, a key test of this replication was whether the framework could identify that it was supposed to use holding companies of the primary dealers instead of the primary dealers directly to get the appropriate values for the table. StatsClaw used primary dealer gvkeys instead of the keys belonging to the holding companies of the primary dealers throughout the replication. Perhaps even worse, there was no methodological documentation of this choice and the way we determined this error was through directly inspecting the gvkeys used. Additionally, there were several errors with the gvkeys used including prespecifying them despite user instructions not to do so, a case where the end date preceded the start date and omitted gvkeys of several dealers with values publicly available. Again, what was concerning about these inaccuracies was that we only discovered them through manual inspection rather than documentation in any of the .md files which highlights a lack of transparency in this run. Therefore, unlike the run with `cds_returns` where a more collaborative human intervention could have saved the replication in this case working collaboratively with StatsClaw could easily mislead a human partner. This is why we would classify this StatsClaw run as a complete failure and for replications of a similar complexity level, recommend a human only implementation. Below is a figure that highlights StatsClaw's performance versus the actual figures in the paper.

HKM (2017) Replication — Evaluation Summary

He, Kelly & Manela (2017) - Tables 2-3 - US primary dealers' WRDS clean Compustat data

A Table 2 — Primary Dealer Size Ratios vs Comparison Groups

Each cell: actual (published) · 12 columns = 4 items × 3 groups · 10 / 36 within tolerance

	Total Assets			Book Debt			Book Equity			Market Equity		
	BD	Banks	Compust	BD	Banks	Compust	BD	Banks	Compust	BD	Banks	Compust
	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$
1960-2012	0.853 (0.959)	0.497 (0.596)	0.096 (0.240)	0.853 (0.960)	0.499 (0.602)	0.112 (0.280)	0.857 (0.939)	0.470 (0.514)	0.032 (0.079)	0.878 (0.911)	0.252 (0.435)	0.019 (0.026)
1960-1990	0.933 (0.967)	0.745 (0.635)	0.072 (0.286)	0.934 (0.998)	0.746 (0.639)	0.087 (0.305)	0.917 (0.961)	0.729 (0.588)	0.025 (0.095)	0.899 (0.961)	0.194 (0.447)	0.006 (0.015)
1990-2012	0.781 (0.914)	0.272 (0.545)	0.117 (0.202)	0.780 (0.926)	0.275 (0.550)	0.134 (0.240)	0.802 (0.883)	0.236 (0.444)	0.038 (0.058)	0.859 (0.948)	0.304 (0.418)	0.030 (0.039)

B Table 3 Panel A — Level Correlations (1970-2012)

actual (published) · 1 / 15 cells within tolerance

	Market Capital	Book Capital	AEM Leverage
E/P ratio	-0.727 (-0.83)	-0.466 (-0.38)	-0.765 (-0.64)
Unemployment	-0.499 (-0.63)	+0.182 (+0.10)	-0.417 (-0.33)
GDP growth	+0.099 (+0.18)	-0.076 (+0.32)	-0.060 (-0.23)
Fin. Conditions	-0.410 (-0.48)	-0.267 (-0.53)	-0.489 (-0.19)
Mkt Volatility	+0.091 (+0.86)	+0.207 (+0.31)	+0.149 (+0.33)

C Table 3 Panel B — Factor Correlations

AR(1) innovations scaled by lagged η · 8 / 16 cells within tolerance

	Mkt Cap Factor	Book Cap Factor	AEM Lev. Factor
Mkt Excess Return	+0.727 (+0.78)	+0.128 (+0.10)	+0.201 (+0.15)
E/P Growth	-0.164 (-0.73)	-0.156 (-0.18)	+0.096 (+0.18)
Unemployment Grw.	+0.065 (+0.05)	+0.187 (+0.12)	-0.113 (-0.08)
GDP Growth	-0.056 (-0.20)	-0.086 (-0.09)	+0.060 (+0.04)
Fin. Cond. Grw.	-0.345 (-0.38)	-0.176 (-0.29)	-0.053 (-0.06)
Mkt Vol. Growth	-0.442 (-0.49)	-0.087 (-0.18)	-0.060 (-0.08)

Verdict: PASS WITH NOTE — all hard BLOCK gates satisfied · Cells outside tolerance attributed to: pre-1978 WRDS coverage gap · post-consolidation subsidiary mismatch · AEM leverage BOG21 vintage · E/P growth smoothing · foreign dealers excluded

Overall, StatsClaw missed most of the values and given the defects in the methodology we discovered the values that appear correct are likely more due to luck than accuracy. This highlights how some replications are still likely too difficult for the StatsClaw framework to solve effectively.

As with the other replications, the base Claude Code attempt at HKM Tables 2 and 3 was done on a separate Claude account from the StatsClaw run to avoid the data leakage issues described in Section 1. The user prompt was similar in structure to that passed onto StatsClaw:

Replicate Tables 2 and 3 only from He, Kelly & Manela (2017), "Intermediary Asset Pricing: New Evidence from Many Asset Classes," JFE 126: 1–35. Paper is at hkm-paper.pdf. Target repo is project/hkm-replication. WRDS credentials are in ~/.env.

Just like the other base Claude runs, this run also used plan mode, which forced the agent to write out a full implementation plan before generating any code. Any time the agent asked a question, it offered a recommendation, which was always followed. This ensured base Claude Code completed the task without any actual outside help. In response, the agent produced a roughly 350-line PLAN.md file that listed every data source, every SIC code, the FRED series IDs for the Z.1 broker-dealer balance sheet items, the regression specification for the AEM seasonal adjustment, and the AR(1) construction for the innovation factors. Nothing in the StatsClaw run has an equivalent of this file, and the contrast between the two arms is mostly about what gets written down before the code starts running.

Beginning with what worked, the biggest improvement over StatsClaw was the way base Claude handled the primary dealer mapping. As we mentioned earlier in this section, the central trap of this replication is that the NY Fed's list names the trading subsidiaries (e.g., "Bear, Stearns & Co., Inc."), while Tables 2 and 3 require the publicly traded U.S. holding companies that owned them. StatsClaw skipped this distinction and used the GVKEYs CRSP returned for the dealer names directly, with no documentation. Base Claude instead produced a curated `pd_to_permno_map.csv` file with 86 rows. Each row carries the original dealer name, the parent holding company, CRSP PERMNO and PERMCO, the Compustat GVKEY, an effective date window, a confidence flag (high, medium, low, or `foreign_excluded`), and a note recording the fuzzy-match score against CRSP's company-name field. Of the 86 rows, 36 are high-confidence, 5 are medium-confidence, 1 is low-confidence (Drexel Burnham Lambert, which has no clean CRSP listing), and 44 are flagged `foreign_excluded` for entities like Deutsche Bank, UBS, Nomura, and Credit Suisse where we cannot replicate the Datastream coverage that the paper used.

Additionally, before computing the actual tables, base Claude cross-checked its reconstructed intermediary capital series against the version that Zhiguo He posts on his data page. Our U.S.-only `market_capital_ratio` correlates +0.96 with the published `intermediary_capital_ratio` across 176 quarters of overlap, and our `market_capital_factor` correlates +0.90 with the published risk factor. What this tells us is that the underlying intermediary capital series is mostly right, and the places where Tables 2 and 3 miss the published values reflect downstream construction choices rather than a broken dealer mapping. StatsClaw performed no equivalent cross-check.

Turning to the actual table results, Table 2 (summarized in the figure below) captures the shape of the published table but runs systematically high in most cells. Of the 36 cells, 12 are within five basis points of the published value, 20 are within fifteen basis points, and 4 miss by more than that. The broker-dealer column over the 1960 to 1990 sub-period is basically exact, with the total assets, book debt, and book equity ratios all matching the paper to within a tenth of one percent. The worst cells sit in the Market Equity by Banks column for the early sub-periods, where our ratios are 0.21 and 0.27 above the paper. The reason for the systematic upward bias traces back to a choice the plan locked in before execution: the denominator for each comparison group was defined as the union of the SIC-defined comparison group and the set of primary dealer parents, which forces the ratio to be at most

one by construction. The paper appears to use the SIC-defined group on its own, without that union. The agent also widened the bank SIC range mid-run, from 6020 to 6029 plus 6712 to 6000 to 6199 plus 6712, in an effort to pull the Banks column closer to the paper. Both choices are written down in the plan and the validation report, so a reviewer can find them, but neither was checked against the paper text before being committed to, and we believe both are wrong.

Table 3 produces a more mixed result. The macro variables against the market capital ratio in Panel A and the market capital factor in Panel B come out close to the paper. The correlation of E/P with market capital is -0.69 versus -0.83 in the paper, NFCI is -0.41 versus -0.48, the market capital factor against market excess return is +0.82 versus +0.78, and the market capital factor against volatility growth is -0.47 versus -0.49. Every sign is correct and every magnitude is within fifteen basis points. The book capital row, however, is where the replication breaks. Our correlation between market and book capital ratios is +0.28 where the paper has +0.50, and the correlation between book capital and AEM leverage is +0.53 where the paper has -0.07, a clean sign reversal. The most likely culprit is the book equity definition. Both the plan and the code use Compustat's CEQQ field as book equity, while the standard construction in the asset pricing literature adjusts for preferred stock by adding back the liquidating, redemption, or par value of preferred shares (whichever is available). The plan's risks section did not flag this as a possible source of error, and the issue made it all the way to the final table without being caught.

The plan's "Decisions already locked in" section explicitly stated that the Panel A row labelled "AEM leverage" should report the implied capital ratio (one over the leverage), not the raw leverage. The Table 3 code as it ended up running uses raw leverage instead, with an in-line comment explaining that the sign of the published +0.42 correlation with market capital "confirms" that the row is leverage and not implied capital. What happened here is that the agent reversed its own pre-committed methodology mid-execution by reading the published number off the paper and fitting its formula to match. The decision is documented in both the plan and the validation report, so a careful reader would notice it, but for autonomous replication of papers whose published values are partially visible to the agent, this is a real concern. Below is a figure that summarizes the per-cell comparison for Tables 2 and 3.

HKM (2017) Replication — Base Claude Code Evaluation Summary

A Table 2 — Primary Dealer Size Ratios vs Comparison Groups
Each cell: replicated (published). 12 pass · 20 close · 4 miss (of 36)

	Total Assets			Book Debt			Book Equity			Market Equity		
	BD	Banks	Cmpust	BD	Banks	Cmpust	BD	Banks	Cmpust	BD	Banks	Cmpust
1960-2012	0.984 (0.959)	0.636 (0.596)	0.344 (0.240)	0.985 (0.960)	0.638 (0.602)	0.391 (0.280)	0.967 (0.939)	0.604 (0.514)	0.140 (0.079)	0.968 (0.911)	0.645 (0.435)	0.137 (0.026)
1960-1990	0.996 (0.997)	0.711 (0.635)	0.397 (0.266)	0.997 (0.998)	0.712 (0.639)	0.455 (0.305)	0.989 (0.988)	0.680 (0.568)	0.170 (0.095)	0.988 (0.961)	0.714 (0.447)	0.167 (0.015)
1990-2012	0.970 (0.914)	0.548 (0.543)	0.278 (0.202)	0.972 (0.916)	0.550 (0.550)	0.314 (0.240)	0.942 (0.883)	0.515 (0.444)	0.102 (0.058)	0.942 (0.848)	0.551 (0.419)	0.095 (0.039)

B Table 3 Panel A — Level Correlations (1970-2012)
0 pass · 3 close · 1 miss · 1 sign wrong (of 5 cells with published reference)

	Market Capital	Book Capital	AEM Leverage
Market Capital	1.00		
Book Capital	+0.28 (+0.50)	1.00	
AEM Leverage	+0.52 (+0.42)	+0.53 (-0.07)	1.00
E/P ratio	-0.69 (-0.83)	-0.57	-0.68
Unemployment	-0.64	+0.14	-0.42
GDP growth	+0.35	+0.81	+0.84
Fin. Conditions	-0.41 (-0.48)	-0.40	-0.45
Mkt Volatility	-0.00	+0.16	+0.15

C Table 3 Panel B — Factor / Growth Correlations
2 pass · 1 close · 0 miss · 0 sign wrong (of 3 cells with published reference)

	Mkt Cap Factor	Book Cap Factor	AEM Lev. Factor
Mkt Cap Factor	1.00		
Book Cap Factor	+0.22	1.00	
AEM Lev. Factor	-0.00	-0.27	1.00
Mkt Excess Return	+0.82 (+0.78)	+0.07	+0.04
E/P Growth	-0.61 (-0.75)	-0.21	-0.01
Unemployment Gro.	+0.01	+0.20	-0.04
GDP Growth	+0.07	-0.09	+0.02
Fin. Cond. Gro.	-0.36	-0.24	-0.04
Mkt Vol. Growth	-0.47 (-0.49)	-0.08	-0.19

Pass |diff| < 0.05 Close 0.05 ≤ |diff| < 0.15 Miss |diff| ≥ 0.15 Sign wrong No published reference

Replication restricts primary dealers to US-listed holding companies (no Datastream coverage for foreign-parent dealers). Table 2 denominators use the SIC-defined comparison group u PD parents — a deviation from the paper that biases ratios upward in nearly every cell. Panel A AEM row reports raw leverage after the agent flipped its own pre-committed choice to match the sign of the published +0.42 correlation with market capital.

Overall, the base Claude Code HKM run is the opposite of what we saw in Cases 1 and 2. StatsClaw was the better framework when the task was mostly about finding and cleaning a series already published somewhere, where the adversarial verification loop could do most of the work. For this case, where the replication actually requires methodological judgment about which firms count as primary dealers, which book equity definition the paper uses, and which formula sits behind which table row, the plan-first workflow turned out to be more useful. Neither framework produced an exact replication of Tables 2 or 3, which is consistent with the Compustat data-vintage problem we raised at the start of this section. The difference is that base Claude's deliverable is something a researcher can read, audit, and intervene on, while StatsClaw's deliverable, as the previous section describes, is not. The right choice of agentic framework appears to depend on where the hard part of the replication actually lies: adversarial verification works when the answer is mostly in the data, and plan-first transparency works better when the answer requires methodological judgment that has to stay visible to the human in the loop.

6. Conclusion and Limitations

To conclude, in order to test the replication abilities of StatsClaw on financial topics, we prompted Claude Code with and without the StatsClaw framework to recreate three cases. The first case involved replicating zero coupon yield data using the methodology established in *Gurkaynak, Refet S., Brian Sack, and Jonathan H. Wright. 2007*. This case was seen as our “easy” case because there exists a publicly available dataset to use and the methodology was, relative to our other cases, straightforward. Our second case centered on constructing a dataset of monthly credit default swap returns and then using that data to create different styles of portfolios using the methods from *He, Kelly, and Manela (2017)* and *Palhares, Diogo (2013)*. This was seen as our “medium” difficulty case because the agents had to pull data from multiple sources and correctly apply methodology from two separate papers. Our last case focused on replicating tables 2 and 3 from *He, Kelly, and Manela (2017)*. We viewed this as our most difficult case because data had to be pulled from several sources, the agents had to correctly identify that they needed to map the subsidiaries to the holding company, and, even after all of those hurdles, a true replication was not possible because of changes to the WRDS database. Prior to running the experiment we expected StatsClaw to perform evenly with Claude Code for the easier test but then gradually outperform it as the cases got more difficult. To our surprise, StatsClaw outperformed Claude Code on the easier case but underperformed Claude Code for the more difficult cases. The following paragraph will explore the potential reason we have identified for this occurring.

Our main ex-post theory as to why Claude Code outperformed StatsClaw for the more difficult replications is that the task was so complicated that as the agents were communicating with each other the context window was becoming too large that the agents began hallucinating. This idea is similar to how in Matt Olson’s lecture the portfolio managed by the agent and sub-agent underperformed because the overly verbose agent was feeding too many tokens into the context window of the sub agent. This also ties into the idea discussed in Homework 3 where even though the agent had RAG abilities, the raw SEC data was too large for the context window, leading to inaccurate identifications. An overfilled context window aligns well with the two main pitfalls of StatsClaws approach to the replication. Perhaps because the context window of the planner agent was so overfilled with content it missed that *He, Kelly, and Manela (2017)* use the holding companies of the primary dealers instead of the primary dealers themselves. Similarly for the medium case, it could have missed that it needed data from the three sources instead of just WRDS. This idea is supported by how the *spec.md* file for these replications were far larger than the *spec.md* file for the yield curve replication, with the CDS replication being nearly three times as large and the *He, Kelly, and Manela (2017)* replication being nearly six times as large. On top of this, the other significant flaw of the *He, Kelly, and Manela (2017)* run was the hallucination of inaccurate gvkey codes which in addition to being incorrect, also violated the specific instructions to identify them independently. Again, this mistake occurred in the *spec.md* file signaling that something was causing the planner agent to make these errors. Therefore, our best post-run hypothesis for why StatsClaw underperformed the more difficult cases, is that the adversarial agents were overloading each others’ context windows, particularly, the planner agent’s, leading to buried important context, hallucinations, and violated instructions.

While these results are both interesting and surprising, there are clear limitations to the conclusions we can draw from them. The first is the small sample size. With only three cases it is completely possible that the results we got were due to chance instead of an overarching pattern. It is also essential to acknowledge AI's response to prompts is a stochastic process. Had we been able to run the same prompt on different Claude accounts, the results may have been completely different. Additionally, there were several subjective components to the judgment and classification of the cases, potentially making our results biased. The classification of the cases as easy, medium, and hard, was in the end, based on our informed opinions, instead of a strict rule or cut-off. On top of this, because the first case was the only case that could be evaluated quantitatively and based on binary choices, the other cases all involved a level of subjective judgment. Furthermore, while we spent significant time crafting our prompts to be neutral and similar across cases, even using AI to tune their neutrality, it is possible that small changes in the wording could produce drastically different results. Therefore, because of the level of user involvement we had several unavoidable opportunities to bias our results. These limitations highlight the inherent variance and bias in our study.

In the end, despite the lack of generalizability, we believe this experiment yielded results both interesting and useful for using AI to replicate financial literature. Continuing to evaluate how and when to best use AI frameworks, especially as AI agents rapidly develop, is essential. The pattern we observed, of StatsClaw being more helpful on simpler tasks and more hurtful on harder ones, is the kind of finding that becomes meaningful only when replicated across more cases. Until that work is done, our results are best read as a case study suggesting that the value of StatsClaw in financial replications may be conditional rather than monotonic.

7. Works Cited

Bejarano, Jeremiah. 2026. "Homework 3: Benchmarking RAG with SEC Filings." *FINM 33200: Generative and Agentic AI for Finance*, Spring 2026, University of Chicago. <https://finm-33200.github.io/HW3.html>.

FTSFR. n.d. *cds_returns*. GitHub repository. https://github.com/ftsfr/cds_returns.

FTSFR. n.d. *fed_yield_curve*. GitHub repository. https://github.com/ftsfr/fed_yield_curve.

Gürkaynak, Refet S., Brian Sack, and Jonathan H. Wright. 2007. "The U.S. Treasury Yield Curve: 1961 to the Present." *Journal of Monetary Economics* 54 (8): 2291–2304. <https://doi.org/10.1016/j.jmoneco.2007.06.029>.

He, Zhiguo, Bryan Kelly, and Asaf Manela. 2017. "Intermediary Asset Pricing: New Evidence from Many Asset Classes." *Journal of Financial Economics* 126 (1): 1–35. <https://doi.org/10.1016/j.jfineco.2017.08.002>.

Olson, Matt. 2026. "Systematic Research Agents: Automated Backtest Research with Claude Code." Presentation, University of Chicago, May 11, 2026.

Palhares, Diogo. 2013. "Cash-Flow Maturity and Risk Premia in CDS Markets." PhD diss., University of Chicago. Working paper available via AQR Capital Management. <https://www.aqr.com/Insights/Research/Working-Paper/Cash-Flow-Maturity-and-Risk-Premia-in-CDS-Markets>.

8. AI Usage Statement

We used Claude to write the base Claude replication code and used Claude to generate the StatsClaw replications after the framework was manually drafted by a human. The accuracy of these outputs was the key part of our project and were evaluated both quantitatively and qualitatively. Lastly, we also used AI to help effectively present our results as well as evaluate the comprehension and continuity of the final write-up.